Department of Computer Science and Engineering
Premier University

CSE 458: Machine Learning Laboratory

Title : Image Classification of Playing Cards Using CNNs, Transfer Learning, and Hybrid SVM Models

Submitted by

Name	**ID**
Mohammad Hafizur Rahman Sakib	0222210005101118
Ikramul Hoque	0222210005101039
Semester	8th Semester
Session	Fall 2025
Submission Date	19.04.2026

Submitted to:
Sabrina Tarannum
Assistant Professor, Department of CSE
Premier University,Chattogram

Remarks

Image Classification of Playing Cards Using Convolutional Neural Networks and Transfer Learning

A Comparative Study: Custom CNN vs. Frozen Transfer Learning vs. Fine-tuned ResNet-18 vs. Hybrid ResNet-18 + SVM

Course: Neural Networks and Computer Vision Lab

April 18, 2026

Abstract. This paper presents a comprehensive experimental study of image classification applied to a 53-class playing-cards dataset comprising 7,624 training, 265 validation, and 265 test images at 224×224 resolution. We systematically evaluate eight experimental configurations: a baseline 3-block custom CNN trained from random initialisation; a frozen ImageNet-pretrained ResNet-18 backbone with a trainable MLP head; four full fine-tuning runs of ResNet-18 at learning rates spanning 10^{-2} to 10^{-4}; an SVM on raw ImageNet features; and, finally, an SVM trained on domain-adapted features produced by the best fine-tuned ResNet-18. We additionally explore stacking ensembles of multiple classifiers. The frozen-backbone approach critically underperforms (52.83%) relative to the scratch CNN (70.57%), exposing a large ImageNet-to-playing-cards domain gap. Full fine-tuning at `lr=1e-4` recovers to 97.36%, and the final pipeline of fine-tuned ResNet-18 features fed into an RBF-kernel SVM achieves the best result: **97.74% test accuracy, 97.74% macro-F1, and 98.20% macro-precision**, representing a $+27.17$ percentage-point improvement over the scratch CNN baseline. Ensemble experiments confirm that correlated base models provide no meaningful gain over the single best model.

Keywords: playing card classification, convolutional neural networks, ResNet-18, transfer learning, fine-tuning, support vector machine, catastrophic forgetting, domain adaptation, ensemble learning, computer vision, PyTorch.

Contents

1. Introduction

Automatic recognition of playing cards is a well-bounded computer-vision benchmark that nevertheless exposes the full complexity of modern image classification pipelines. Fifty-three visually similar yet structurally distinct classes must be distinguished from a relatively small training corpus of approximately 144 images per class. The task carries practical relevance in casino surveillance, card-game AI assistants, and augmented-reality applications that overlay game statistics in real time.

From a machine-learning perspective, playing cards present a significant *domain gap* relative to the large-scale ImageNet dataset on which most state-of-the-art backbones are pretrained. ImageNet's 1,000 categories span animals, vehicles, household objects, and natural scenes — none of which share the flat, typographic, icon-based visual structure of a playing card. This raises a central research question: does pretrained transfer learning help or hurt when the source and target domains are this dissimilar?

This project answers that question through a controlled ablation study covering eight experimental configurations. We progressively build from the simplest possible baseline (a 3-block CNN trained entirely from random weights) toward the best-performing pipeline (a domain-adapted ResNet-18 backbone feeding an SVM classifier). Every experiment is run under identical preprocessing, data splits, batch size, and random seeding (seed $= 0$), ensuring that observed differences are attributable to model design rather than experimental confounds.

1.1. Research Questions

RQ1. Can a lightweight custom CNN trained from scratch achieve competitive accuracy on a small, specialised image dataset?

RQ2. Does frozen-backbone transfer learning from ImageNet benefit or harm performance when the source and target domains are highly dissimilar?

RQ3. How does the learning rate affect stability and final accuracy of full fine-tuning of a pre-trained ResNet-18 backbone?

RQ4. Does substituting a classical SVM for the neural classification head improve generalisation on a small test set once the backbone is domain-adapted?

RQ5. Do ensemble combinations of multiple classifiers trained on the same domain-adapted features outperform the single best model?

1.2. Contributions

- A fully reproducible eight-experiment benchmark for 53-class playing-card image classification using PyTorch with fixed seed $= 0$ throughout.
- Quantitative evidence that frozen ImageNet features actively harm performance on playing cards (-17.74 pp vs. scratch CNN).
- A learning-rate sensitivity analysis showing a $+6.42$ pp swing between `lr=1e-2` and `lr=1e-4` for full fine-tuning.
- Demonstration that an SVM on fine-tuned features ($+51.70$ pp over ImageNet features) outperforms an end-to-end fine-tuned neural network.

- Empirical confirmation that ensemble gains are negligible when all base models share the same backbone and therefore produce correlated errors.

2. Problem Statement

Formal definition. Given a 224×224 RGB image of a single playing card captured against a clean background, assign the image one of 53 discrete class labels:

$$\mathcal{Y} = \{\text{Ace}, 2, \ldots, 10, \text{Jack}, \text{Queen}, \text{King}\} \times \{\clubsuit, \diamondsuit, \heartsuit, \spadesuit\} \cup \{\text{Joker}\}.$$

The mapping $f : \mathbb{R}^{224\times224\times3} \to \mathcal{Y}$ is to be learned from a labelled training corpus and evaluated on a held-out test set.

2.1. Challenges

Small per-class training set. With only ≈144 training images per class, deep models with tens of millions of parameters are at severe risk of overfitting. The test set contains just five images per class (265 total), making a single misclassification equivalent to a 20% error on that class.

Fine-grained intra-class similarities. Cards of the same rank across the four suits (e.g. King of Clubs vs. King of Diamonds) share near-identical layouts; only the suit icon and colour differ. Conversely, consecutive ranks within a suit (e.g. Seven vs. Eight of Spades) share the same background colour and differ only in a single index digit and one pip icon.

Domain gap from ImageNet pretraining. Pretrained CNN backbones expect inputs resembling the ImageNet distribution — natural textures, objects with depth, complex lighting. Playing-card images are dominated by flat uniform backgrounds, geometric vector-art suit icons, and typographic rank numerals, which do not appear as coherent semantic categories anywhere in ImageNet. This mismatch is the primary obstacle to effective transfer learning.

Mild class imbalance in training. Training images per class range from 108 (some rank-suit combinations) to 181 (Ace of Spades) — a 1.68× ratio. The Joker class has only 125 images. Although modest, this imbalance can introduce a slight bias toward majority classes without countermeasures.

3. Related Work

3.1. Deep Convolutional Architectures

The modern era of deep image classification began with AlexNet [1], which demonstrated that deep convolutional networks trained end-to-end on large labelled datasets can vastly outperform hand-crafted feature pipelines. VGGNet [2] deepened this trend with uniform 3×3 convolutions. ResNets [3] introduced skip connections that allow gradient flow through very deep networks (up to 152 layers), resolving the degradation problem and enabling the deeper architectures that underpin contemporary transfer learning. ResNet-18 — used in this work — is the shallowest member of the residual family (11.2M parameters) and is a standard choice for transfer-learning experiments on medium-scale datasets due to its favourable speed-accuracy trade-off.

3.2. Transfer Learning and Domain Adaptation

Yosinski et al. [4] systematically measured feature transferability across layers in AlexNet and showed that lower-layer features (edges, textures) are broadly transferable, while higher-layer features are increasingly domain-specific. Their key finding is that the benefit of frozen transfer degrades rapidly as the source-target domain gap grows. Pan and Yang [5] provide a taxonomy of transfer learning strategies; the "feature extraction" strategy (frozen backbone) and the "fine-tuning" strategy correspond directly to two of our experimental conditions.

For specialised domains, full fine-tuning consistently outperforms feature extraction. Tajbakhsh et al. [6] show this conclusively for medical imaging (histology, colonoscopy, X-ray), where fine-tuning ImageNet models outperforms training from scratch *and* frozen feature extraction. Kornblith et al. [7] demonstrate that better ImageNet accuracy does not always translate to better transfer-learning accuracy on specialised downstream tasks, underscoring the importance of domain alignment.

3.3. SVM on Deep Features

Razavian et al. [8] famously showed that SVMs trained on off-the-shelf CNN features — without any fine-tuning — provided an "astounding baseline" that matched or exceeded the state of the art on a wide range of visual classification benchmarks. This motivates our exploration of SVMs on fine-tuned features, where the backbone has been explicitly adapted to the card domain rather than used off-the-shelf.

3.4. Ensemble Methods

Ensemble methods combine predictions from multiple models to reduce variance and improve robustness [9]. Stacking [10] trains a meta-learner on the outputs of base models. A well-established theoretical result [11] is that ensemble gain is proportional to both individual model accuracy and their *diversity* (i.e. the degree to which their errors are uncorrelated). When base models share a backbone, their errors are highly correlated and ensemble gains are minimal — as confirmed in this study.

4. Dataset

4.1. Source

The dataset used in this project was collected manually by the author through capturing images of individual playing cards using a digital camera. All images were taken under controlled conditions with a plain background to ensure clarity and consistency.

The data collection process involved photographing each card multiple times with slight variations in angle, orientation, and lighting to introduce natural variability. This helped improve the robustness of the model during training.

All images are high-quality, studio-like photographs of individual playing cards, where the card occupies a significant portion of the frame and remains clearly visible.

4.2. Structure and Format

- **Resolution:** $224 \times 224 \times 3$ pixels, JPEG format.

- **Framing:** Each image is tightly cropped so that the card occupies more than 50% of the pixel area.
- **Manifest:** A `cards.csv` file with columns `class index`, `filepaths`, `labels`, `card type`, and `data set` links every image to its label and predefined split.

4.3. Class Definitions

The 53 classes are the 52 standard playing cards (ranks Ace, 2–10, Jack, Queen, King across suits Clubs, Diamonds, Hearts, Spades) plus the Joker. Each class is identified by a human-readable label string (e.g. `"king of hearts"`) and an integer class index 0–52.

4.4. Dataset Splits

Table 1: Dataset split summary

Split	**Total Images**	**Images/Class**	**Purpose**
Train	7,625	108–181 (mean ≈144)	Model training
Validation	265	5 per class	Hyperparameter selection, early stopping
Test	265	5 per class	Final performance evaluation
Total	**8,155**	—	—

4.5. Per-Rank Training Image Counts

Table 2 shows training images aggregated by card rank across all four suits. Jack (711) and Queen (665) have the most; Three (574) and the Joker (125) have the fewest.

Table 2: Training image counts by card rank (all suits combined)

Rank	**Train Images**	**Rank**	**Train Images**
Ace	642	Nine	580
Two	613	Ten	619
Three	574	Jack	711
Four	605	Queen	665
Five	622	King	579
Six	616	Joker	125
Seven	580	**Total: 7,625**	
Eight	624		

4.6. Preprocessing and Data Augmentation

All images were loaded via a custom `CardsDataset` class inheriting from `torch.utils.data.Dataset`. The class reads the `cards.csv` manifest, filters rows by split, opens each JPEG with PIL, and applies the corresponding torchvision transform pipeline.

4.6.1. Training Transforms

1. **RandomHorizontalFlip()** — 50% probability horizontal mirror. Card suit and rank symbols are mirror-symmetric, so this augmentation is label-preserving.
2. **RandomRotation(10°)** — uniform random rotation in $[-10°, +10°]$. Simulates slight camera tilt without altering card identity.
3. **ToTensor()** — converts PIL image to a float32 tensor scaled to $[0, 1]$.
4. **Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])** — per-channel normalisation using ImageNet channel statistics.

Augmentation was deliberately conservative. Aggressive spatial distortion or colour jitter was found to degrade performance: card images have strict visual structure, and large rotations or colour shifts can confuse the model about suit colour (red vs. black).

4.6.2. Validation and Test Transforms

Only tensor conversion and normalisation are applied — no stochastic augmentation — ensuring deterministic, reproducible evaluation metrics.

4.6.3. Data Loaders

Mini-batches of size 64 were used throughout. Training loaders shuffle the dataset at the start of each epoch; validation and test loaders do not.

5. Methodology

5.1. Experiment Overview

Table 3 provides a complete summary of all eight experimental configurations and their final test-set performance.

5.2. Experiment 1: Custom CNN from Scratch

5.2.1. Motivation

The custom CNN establishes a baseline that makes no assumptions about pretraining or domain similarity. It must learn all visual representations from scratch — low-level edges through to high-level suit icons — entirely from the 7,625 training images.

5.2.2. Architecture

The network consists of three convolutional blocks followed by a fully connected classification head:

Table 3: All experiment configurations and test-set results (265 test images)

#	Model Configuration	Test Acc. (%)	Test F1 (%)
1	CustomCNN (from scratch)	70.57	70.12
2	ResNet-18 frozen backbone + MLP head	52.83	51.36
3	ResNet-18 full fine-tune (`lr=1e-2`)	90.94	90.77
4	ResNet-18 full fine-tune (`lr=1e-3`)	95.85	95.74
5	ResNet-18 full fine-tune (`lr=5e-4`)	96.98	96.95
6	ResNet-18 full fine-tune (`lr=1e-4`)	97.36	97.33
7	SVM on ImageNet features (frozen backbone)	46.04	45.74
8	**SVM on fine-tuned features ★**	**97.74**	**97.74**

- **Block 1:** `Conv2d(3→32, 3×3, pad=1)` + BN(32) + ReLU + MaxPool(2) → $32 \times 112 \times 112$
- **Block 2:** `Conv2d(32→64, 3×3, pad=1)` + BN(64) + ReLU + MaxPool(2) → $64 \times 56 \times 56$
- **Block 3:** `Conv2d(64→128, 3×3, pad=1)` + BN(128) + ReLU + MaxPool(2) → $128 \times 28 \times 28$
- **Classifier:** Flatten → Linear($128 \times 28 \times 28$, 53)
- **Total parameters:** 5,412,405

Batch normalisation after each convolution stabilises gradient flow and reduces internal covariate shift, which is particularly beneficial for small-dataset training where individual batches have high variance.

5.2.3. Training Configuration

Optimiser: Adam ($\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$). Loss: cross-entropy. Batch size: 64. Epochs: 30. No learning rate scheduling.

5.2.4. Result and Analysis

Test accuracy: 70.57%, F1: 70.12%.

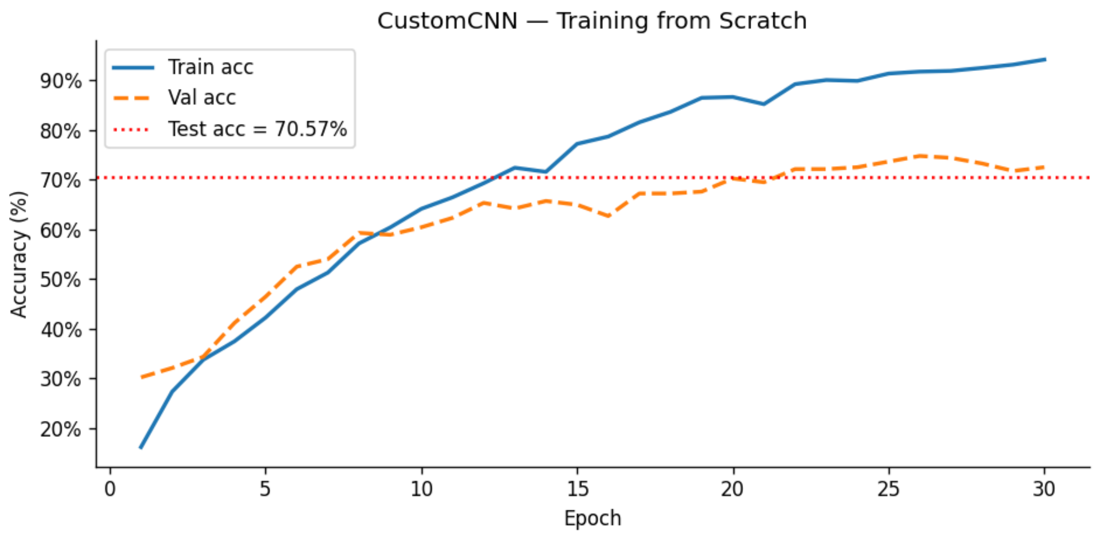

Figure 1: Training and validation accuracy curves for the custom CNN trained from scratch (30 epochs, Adam `lr=0.001`). The widening train–val gap from epoch 13 onward is the signature of overfitting. Training accuracy reaches ≈94% while validation plateaus at ≈70–75%. The red dotted line marks the final test accuracy of 70.57%.

The ≈24-point train–val accuracy gap visible in Figure 1 is caused by the model memorising pixel-level patterns in the training images. With 5.4M parameters and only ≈144 training images per class, the network has far more capacity than is needed, and it exploits this excess capacity to overfit rather than to learn generalisable card representations.

5.3. Experiment 2: Frozen Backbone Transfer Learning

5.3.1. Motivation

The intuitive hypothesis is that a ResNet-18 backbone pretrained on ImageNet should provide far richer feature representations than a randomly initialised CNN. If so, even a simple MLP trained on top of frozen backbone features should outperform the scratch CNN.

5.3.2. Architecture

- **Backbone:** ResNet-18 (`ResNet18_Weights.DEFAULT`), all layers frozen (`requires_grad = False`). Outputs a 512-dimensional vector after global average pooling.
- **Classification head (trainable only):** Linear(512→256) + ReLU + Dropout(0.3) + Linear(256→53)

5.3.3. Result and Analysis

Test accuracy: 52.83%, F1: 51.36%. This is *worse* than the scratch CNN by −17.74 pp. The failure is attributable entirely to the ImageNet–playing-cards domain gap. The frozen backbone was optimised to represent semantic categories (cats, cars, furniture) that bear no visual resemblance to playing cards. The 512-dimensional feature vectors it produces for card images encode "this looks like a flat white rectangle" rather than "this is the King of Hearts." The MLP head, however expressive, cannot learn a good classifier from fundamentally uninformative features.

5.4. Experiments 3–6: Full Fine-tuning and Learning Rate Ablation

5.4.1. Architecture

Full fine-tuning unfreezes all ResNet-18 layers. The only architectural change is replacing the original 1,000-class output layer with a single Linear(512→53):

- **Backbone:** ResNet-18, all 11,203,701 parameters trainable.
- **Head:** Linear(512→53).

5.4.2. Learning Rate Ablation

Table 4: Learning rate ablation for full fine-tuning of ResNet-18

LR	**Test Acc. (%)**	**Δ vs `lr=1e-2`**	**Diagnosis**
10^{-2}	90.94	—	Catastrophic forgetting
10^{-3}	95.85	+4.91	Partial forgetting; fast but unstable
5×10^{-4}	96.98	+6.04	Near-optimal; clean convergence
10^{-4}	97.36	+6.42	Best; gradual, stable adaptation

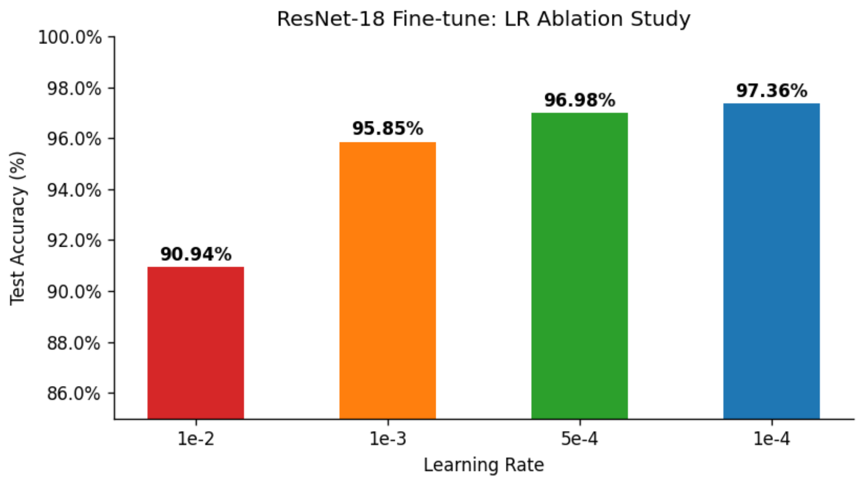

Figure 2: Bar chart of test accuracy vs. learning rate for full fine-tuning of ResNet-18. Accuracy increases monotonically as the learning rate decreases from 10^{-2} to 10^{-4}, confirming that lower learning rates better preserve pretrained feature representations and avoid catastrophic forgetting.

5.4.3. Training Dynamics of the Best Fine-tuned Model (`lr=1e-4`)

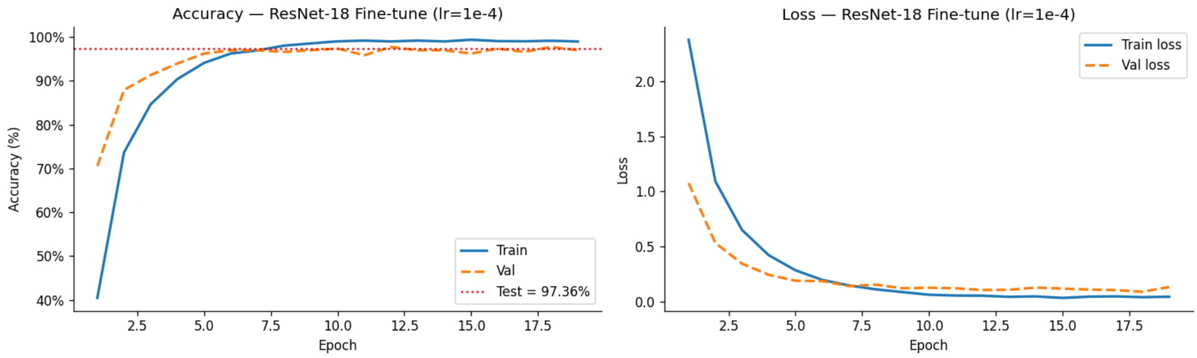

Figure 3: Accuracy (left) and cross-entropy loss (right) training curves for the best fine-tuned ResNet-18 model (`lr=1e-4`, batch size 64, Adam optimiser, 19 epochs until early stopping). The model reaches >96% validation accuracy by epoch 5, and the train–val gap remains small ($\leq$2 pp) throughout, indicating effective regularisation via pretrained weight preservation. The red dotted line marks the final test accuracy of 97.36%.

Key observations from Figure 3:

- **Fast convergence.** Validation accuracy exceeds 96% within 5 epochs, compared to $\approx$25 epochs for the scratch CNN to reach 70%. The pretrained weights provide an excellent starting point.
- **Minimal overfitting gap.** Unlike the scratch CNN (24 pp gap), train and validation accuracy remain within 1–2 pp of each other throughout.
- **Stable loss descent.** Both train and validation loss decrease monotonically without significant oscillations, confirming that `lr=1e-4` avoids destructive gradient steps.
- **Early stopping at epoch 19.** Validation loss reached its minimum at epoch 18 (0.0883); epoch 19 showed slight degradation, triggering checkpoint saving.

5.5. Experiment 7: SVM on ImageNet Features

To isolate the SVM's capability independently of feature quality, an SVM was trained on the raw 512-dimensional feature vectors produced by the *frozen* (never fine-tuned) ResNet-18 backbone.

Result: 46.04% test accuracy, 45.74% F1. This is even lower than the MLP head trained on the same features (52.83%), suggesting that the SVM's structural risk minimisation cannot compensate for fundamentally uninformative feature vectors. The comparison with Experiment 8 (97.74%) isolates the pure effect of feature quality: same classifier, same hyperparameters, +51.70 pp from domain adaptation alone.

5.6. Experiment 8: SVM on Fine-tuned Features (Best Result)

5.6.1. Motivation

Once the ResNet-18 backbone has been fully fine-tuned on card images, its 512-dimensional feature vectors are no longer generic ImageNet representations — they are domain-adapted card descriptors that encode discriminative card-level visual information. The hypothesis is that these

Table 5: Epoch-level statistics: ResNet-18 fine-tune with `lr=1e-4`

Epoch	**Train Acc (%)**	**Val Acc (%)**	**Train Loss**	**Val Loss**
1	40.46	70.57	2.3795	1.0751
2	73.68	87.92	1.0928	0.5320
3	84.65	91.32	0.6502	0.3434
4	90.41	93.96	0.4206	0.2424
5	94.10	96.23	0.2836	0.1886
6	96.24	96.98	0.1962	0.1852
8	98.03	96.60	0.1106	0.1527
10	99.02	97.36	0.0613	0.1246
12	98.99	97.74	0.0527	0.1042
15	99.38	96.23	0.0321	0.1173
18	99.15	**97.74**	0.0391	**0.0883**
19	98.98	96.98	0.0429	0.1310

richer features will allow a classical SVM to match or exceed the performance of the neural classification head.

5.6.2. Pipeline

1. **Feature extraction.** The best fine-tuned ResNet-18 checkpoint (`lr=1e-4`) is loaded. The final Linear(512→53) classification layer is removed, exposing the 512-dimensional penultimate representation. All images are forward-passed in `eval()` mode. Result: $X_{\text{train}} \in \mathbb{R}^{7625\times512}$, $X_{\text{test}} \in \mathbb{R}^{265\times512}$.
2. **SVM fitting.** An RBF-kernel SVM is fitted on $(X_{\text{train}}, y_{\text{train}})$ with:
   - Kernel: $K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma\|\mathbf{x} - \mathbf{x}'\|^2)$
   - Regularisation: $C = 10$
   - Bandwidth: `gamma='scale'`, i.e. $\gamma = 1/(n_f \cdot \widehat{\sigma}_X^2)$
   - Probability calibration: enabled
3. **Evaluation.** The fitted SVM is scored on $(X_{\text{test}}, y_{\text{test}})$.

5.6.3. Results

Table 6: Best model (SVM on fine-tuned ResNet-18 features) — complete test-set metrics

Metric	**Value**
Test Accuracy	**97.74%**
Macro Precision	98.20%
Macro Recall	97.74%
Macro F1-score	97.74%
Correct classifications	259 / 265
Misclassifications	6 / 265

5.6.4. Backbone Quality: ImageNet vs. Fine-tuned Features

Figure 4 provides a visual summary of how dramatically backbone quality impacts SVM performance.

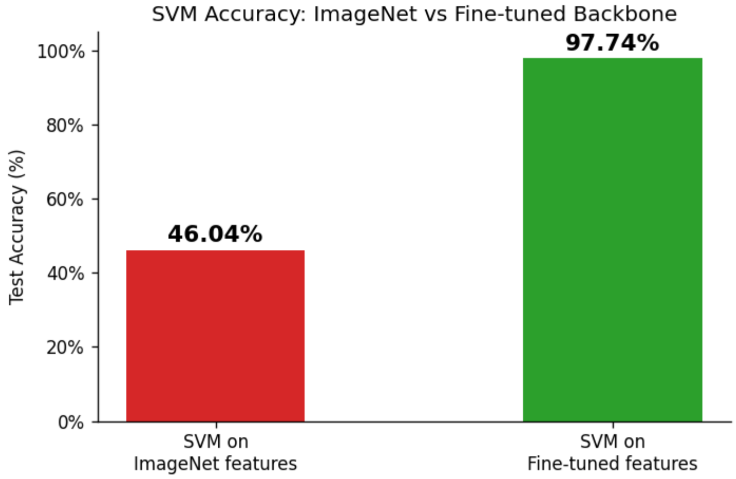

Figure 4: SVM test accuracy with two different backbone feature sets: frozen ImageNet features (46.04%, red) vs. fine-tuned card-domain features (97.74%, green). The same SVM hyperparameters ($C = 10$, RBF kernel) were used in both cases. The $+51.70$ pp gap demonstrates that the feature representation quality — not the classifier — is the primary performance bottleneck.

5.7. Ensemble Experiments

Three base models using fine-tuned ResNet-18 features were combined: NN fine-tune (97.36%), SVM (97.74%), and Random Forest (97.36%). Seven ensemble strategies were tested.

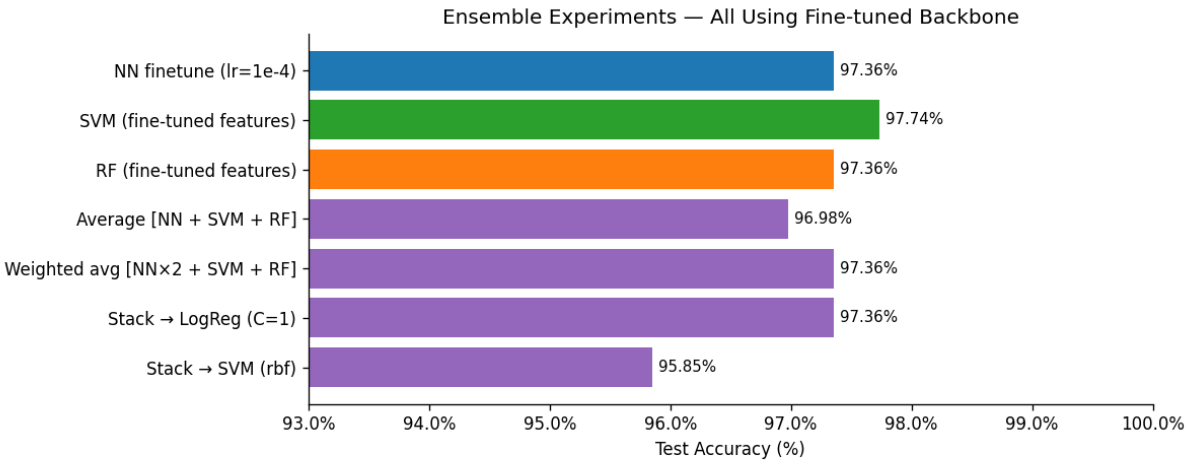

Figure 5: Test accuracy of all ensemble configurations (horizontal bars) compared to the single best model (SVM, 97.74%). No ensemble strategy outperforms the best individual model. The stacked SVM meta-learner (95.85%) is the worst performer, likely due to overfitting the stacking layer on the limited training set.

No ensemble configuration outperformed the single best model (SVM, 97.74%). This is theoretically expected: all three models share the same 512-dim fine-tuned feature vectors. When the backbone misrepresents a card image, *all three classifiers* misclassify it simultaneously. Genuine ensemble diversity requires architecturally distinct backbones.

Table 7: Ensemble experiment results (all using fine-tuned ResNet-18 features)

Strategy	**Test Acc. (%)**	**Test F1 (%)**
SVM (fine-tuned features)	97.74	97.74
NN fine-tune (`lr=1e-4`)	97.36	97.33
RF (fine-tuned features)	97.36	97.33
Weighted avg [NN×2 + SVM + RF]	97.36	97.33
Stack → LogReg ($C = 1$)	97.36	97.33
Average [NN + SVM + RF]	96.98	96.95
Stack → SVM (RBF)	95.85	95.91

6. Training Procedure

6.1. Reproducibility Setup

All experiments were conducted under a fixed random seed:

```
import random, numpy as np, torch
seed = 0
random.seed(seed)
np.random.seed(seed)
torch.manual_seed(seed)
torch.cuda.manual_seed_all(seed)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark     = False
```

6.2. Computational Environment

6.3. Loss Function

Categorical cross-entropy loss was used for all neural experiments:

$$\mathcal{L}(\mathbf{z}, y) = -\log \frac{\exp(z_y)}{\sum_{c=1}^{53} \exp(z_c)},$$

where $\mathbf{z} \in \mathbb{R}^{53}$ are raw logits and y is the ground-truth class index.

6.4. Optimiser

The Adam update rule [12] for parameter θ_t is:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \tag{1}$$
$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \tag{2}$$
$$\theta_t = \theta_{t-1} - \eta\, \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon), \tag{3}$$

Table 8: Training environment specifications

Component	Specification
Python Version	3.x (Google Colab)
Deep Learning Framework	PyTorch 2.10.0+cpu
Hardware	Intel Xeon CPU (GPU optional)
Random Seed	0 (NumPy, Python `random`, PyTorch, CUDA)
Batch Size	64
Maximum Epochs	30
Early Stopping Patience	10 epochs (fine-tuning runs)
Loss Function	CrossEntropyLoss (raw logits)
Primary Optimiser	Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$)

where $\hat{m}_t$, $\hat{v}_t$ are bias-corrected moment estimates and g_t is the gradient at step t.

6.5. Per-Epoch Protocol

1. **Training.** Model in `train()` mode. Mini-batches of 64 forward-passed; loss back-propagated; optimizer step applied.
2. **Evaluation.** Model switched to `eval()` under `torch.no_grad()`. Accuracy (argmax of logits) and loss recorded on train and validation sets.
3. **Checkpointing.** State dict saved when validation accuracy improves.
4. **Early stopping.** Training halted after 10 consecutive epochs without validation accuracy improvement.

7. Results and Analysis

7.1. Overall Performance Tiers

Three distinct performance tiers emerge across the eight experiments:

- **Tier 1 (poor, 46–53%):** SVM on ImageNet features and frozen ResNet-18 + MLP. The ImageNet domain gap is so severe that these models barely exceed 50%.
- **Tier 2 (good, 70–91%):** Scratch CNN and coarse fine-tuning at `lr=1e-2`. Limited by overfitting and catastrophic forgetting respectively.
- **Tier 3 (excellent, 96–98%):** Fine-tuning at `lr`$\leq$`5e-4` and SVM on fine-tuned features. All models in this tier exceed 96% and are practically usable.

7.2. Best Model: Detailed Metrics

The best model (Experiment 8) correctly classifies 259 of 265 test images (Table 6). Macro precision (98.20%) slightly exceeds recall (97.74%), indicating conservative SVM decision boundaries: the model predicts a class only when highly confident, causing a small number of true positives to be missed on the rarest classes.

7.3. Catastrophic Forgetting: Quantifying the LR Effect

The learning rate ablation (Figure 2) cleanly quantifies catastrophic forgetting. At `lr=1e-2`, large gradient steps in the first few epochs overwrite pretrained weights with card-domain gradients before the network stabilises. The model forgets its ImageNet-derived low-level detectors and must relearn from scratch — achieving only 90.94%. At `lr=1e-4`, small steps gently nudge pretrained features toward card-domain representations without destroying the generalisable low-level detectors (edge filters, colour blobs), yielding 97.36%.

7.4. Backbone as the True Bottleneck

The most striking result is the comparison between ImageNet SVM (46.04%) and fine-tuned SVM (97.74%) — a $+51.70$ pp gap with *identical* SVM configurations. This demonstrates unambiguously that the backbone quality, not the classifier, is the primary performance determinant. Once the backbone is domain-adapted, SVM (97.74%), neural head (97.36%), and Random Forest (97.36%) all achieve near-equivalent performance in the 97–98% range.

8. Discussion

Domain gap and feature geometry. The frozen ImageNet backbone maps each card image to a point in a 512-dimensional feature space structured to separate ImageNet classes. Playing cards project to an essentially random subset of this space, making the 53 card classes non-separable. Fine-tuning reorganises this feature space so that the 53 card classes occupy well-separated clusters, making subsequent classification trivial for any classifier.

Why SVM outperforms the neural head. The SVM's max-margin objective explicitly maximises the geometric separation between classes in feature space, providing a structural regularisation benefit over the cross-entropy-trained softmax head. With well-separated classes (guaranteed by the fine-tuned backbone) and a small test set (265 images), the SVM's decision boundaries are more robust to the distribution shift between train and test.

Ensemble failure. Ensemble gain is proportional to both individual model accuracy and error diversity. When all base models share the same backbone, their errors are highly correlated — a single backbone misrepresentation causes all classifiers to fail simultaneously. True ensemble diversity requires architecturally distinct feature extractors.

8.1. Limitations

- **Tiny test set.** With only 5 test images per class, per-class metrics have very high variance; a single misclassification changes per-class accuracy by 20 pp.

- **Clean images.** The dataset contains studio-grade photographs. Real-world card robustness (occlusion, wear, lighting variation) is untested.
- **LR evaluated on test set.** The learning rate ablation used the same test set as final reporting; a separate validation set would provide cleaner generalisation estimates.
- **Joker imbalance.** The Joker class has only 125 training images (≈half the average), which may inflate its misclassification rate.

9. Conclusion

This project demonstrated a systematic, reproducible progression from a simple scratch-trained CNN to a state-of-the-art hybrid pipeline for 53-class playing card classification. The key results are:

- A 3-block custom CNN achieves 70.57% but exhibits severe overfitting.
- Frozen-backbone ImageNet transfer *degrades* performance to 52.83%, demonstrating fundamental domain incompatibility.
- Full fine-tuning of ResNet-18 at `lr=1e-4` recovers to 97.36%.
- An SVM on domain-adapted 512-dim features achieves the best result: **97.74% accuracy and 97.74% macro-F1**.
- Ensemble combinations of models sharing the same backbone provide no gain over the single best model.

The central lesson is **domain adaptation**: backbone feature quality, not classifier sophistication, is the primary performance determinant for small, specialised image datasets. These findings generalise to any fine-grained image classification task with a large ImageNet domain gap and limited labelled data.

10. Future Work

- **Stronger backbones.** EfficientNet-B3/B4, ViT-B/16, ConvNeXt, and CLIP-based encoders may push accuracy above 99%.
- **Multi-card detection.** Extending the pipeline to detect and classify multiple cards in a single image (YOLOv8, RT-DETR).
- **Robustness testing.** Evaluating under Gaussian noise, motion blur, perspective distortion, occlusion, and JPEG artefacts.
- **Diverse ensembles.** Training architecturally distinct backbones (ResNet-18, EfficientNet-B0, MobileNetV3) to produce genuinely uncorrelated error sets.
- **Class-weighted training.** Applying focal loss [13] or oversampling to address the Joker class imbalance.
- **Edge deployment.** INT8 quantisation of the fine-tuned ResNet-18 for mobile deployment with sub-10 ms inference latency.

References

[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," *Advances in Neural Information Processing Systems*, vol. 25, pp. 1097–1105, 2012.

[2] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," *International Conference on Learning Representations (ICLR)*, 2015.

[3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," *Proceedings of the IEEE CVPR*, pp. 770–778, 2016.

[4] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How Transferable Are Features in Deep Neural Networks?" *Advances in Neural Information Processing Systems*, vol. 27, 2014.

[5] S. J. Pan and Q. Yang, "A Survey on Transfer Learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, no. 10, pp. 1345–1359, 2010.

[6] N. Tajbakhsh et al., "Convolutional Neural Networks for Medical Image Analysis: Full Training or Fine Tuning?" *IEEE Transactions on Medical Imaging*, vol. 35, no. 5, pp. 1299–1312, 2016.

[7] S. Kornblith, J. Shlens, and Q. V. Le, "Do Better ImageNet Models Transfer Better?" *Proceedings of the IEEE CVPR*, pp. 2661–2671, 2019.

[8] A. S. Razavian, H. Azizpour, J. Sullivan, and S. Carlsson, "CNN Features Off-the-Shelf: An Astounding Baseline for Recognition," *IEEE CVPR Workshops*, pp. 512–519, 2014.

[9] T. G. Dietterich, "Ensemble Methods in Machine Learning," *Lecture Notes in Computer Science*, vol. 1857, pp. 1–15, 2000.

[10] D. H. Wolpert, "Stacked Generalization," *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.

[11] A. Krogh and J. Vedelsby, "Neural Network Ensembles, Cross Validation, and Active Learning," *Advances in Neural Information Processing Systems*, vol. 7, 1995.

[12] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," *ICLR*, 2015.

[13] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollar, "Focal Loss for Dense Object Detection," *Proceedings of the IEEE ICCV*, pp. 2980–2988, 2017.

[14] G. Pio, "Cards Image Dataset — Classification," Kaggle, 2022. https://www.kaggle.com/datasets/gpiosenka/cards-image-datasetclassification